

Assignment: Numerical Differentiation

Name/ID: 유채은 / 22200476

Date: 25/09/24

Academic Integrity Declaration

Check the statement that applies, and sign your name.

Check	Statement
✓	I have NOT used LLM (ChatGPT, etc), any online materials, or other students' reports/codes to complete this assignment.
	I have NOT completed this assignment entirely on my own.

If you received any assistance **that violates the class policy**, you must explain it in detail. A penalty will be applied depending on the **extent and nature of the assistance**.

Failure to disclose the assistance, you will get Grade F.

Examples:

- *Screenshots of LLM prompt,*
- *Explanation of assistance : who/how/what extent etc*

By signing below, I confirm that I have honestly and truthfully answered the academic integrity declaration.

Date: 2025.09.24

What you need to submit

Submit as a zip file: [Assignment_Differentiation_YuChaeun_22200476.zip](#)

- **Report** (pdf)
- **Src Codes:** (1) [Assignment_Differentiation_YuChaeun_22200476.cpp](#)
 (2) [myNP_22200476.h](#), [myNP_22200476.cpp](#)

Problem

Estimate the velocity and acceleration from datasets of the position of an object.

```
X=[-1 -0.5 0 0.5 1 1.5 2 2.5 3 3.5 4 4.5]
```

```
Y=[-3.632 -0.3935 1 0.6487 -1.282 -4.518 -8.611 -12.82 -15.91 -15.88 -9.402 9.017]
```

0. Preparation: Review tutorial

- [Tutorial: Passing a function/function callback](#)

1. Create a function for numerical 1st order differentiation from a set of discrete data

- With assumption of 3 or more data points, for m dataset
 - 3-Point forward difference: for the first point $x[0]$
 - 2-Point central difference: for $x[1] \sim x[m-2]$
 - 3-Point backward difference: for the last point $x[m-1]$

- Write down a pseudocode for the function

```
// Your Pseudocode
```

```
h = 0.2
```

```
// Three-point forward difference
```

```
dydx(0) = { -3y(0) + 4y(1) - y(2) } / 2h
```

```
// Two-point central difference
```

```
For( i = 1: m-2 )
```

```
dydx[i] = { y(i + 1) - y(i - 1) } / 2h
```

```
end
```

```
// Three-point backward for the last point
```

```
dydx(m - 1) = { y(m - 3) - 4y(m - 2) + 3y(m - 1) } / 2h
```

- Create a function for numerical differentiation from a set of data

```
void gradient1D (double x[], double y[], double dydx[], int m)
```

```
*****
|                                     |
|                               PART 1.                               |
|                                     |
*****
Vector[0] = 8.32
Vector[1] = 4.63
Vector[2] = 1.04
Vector[3] = -2.28
Vector[4] = -5.17
Vector[5] = -7.33
Vector[6] = -8.30
Vector[7] = -7.30
Vector[8] = -3.06
Vector[9] = 6.51
Vector[10] = 24.90
Vector[11] = 48.78
```

```

void gradient1D (double x[ ], double y[ ], double dydx[ ], int m){

    // h = x[i+1] - x[i]

    // Three-point forward difference
    dydx[0] = (-3 * y[0] + 4 * y[1] - y[2]) / (x[2] - x[0]);

    // Two-point central difference
    for (int i = 1; i < m - 1; i++) {
        dydx[i] = (y[i + 1] - y[i - 1]) / (x[i + 1] - x[i - 1]);
    }

    // Three-point backward for the last point
    dydx[m - 1] = (y[m - 3] - 4 * y[m - 2] + 3 * y[m - 1]) / (x[m - 1] - x[m - 3]);
}

```

2. Create a function for numerical 1st order differentiation from the user-defined equation

- Define a function that defines the target equation.

```
double myFunc(const double x)
```

- Create the following function that calls myFunc() to generate dataset and estimate differentiation.

```
void gradientFunc(double func(const double x), double x[], double dydx[], int m)
```

- Check your function with the test equation

$$y=x^3 \quad \text{at } x=0:0.2:4$$

```
*****
|                                     |
|                               PART 2.                               |
|                                     |
*****

y=myFun(xin) = 15.625000

-----
Vector[0] = -0.08
Vector[1] = 0.16
Vector[2] = 0.52
Vector[3] = 1.12
Vector[4] = 1.96
Vector[5] = 3.04
Vector[6] = 4.36
Vector[7] = 5.92
Vector[8] = 7.72
Vector[9] = 9.76
Vector[10] = 12.04
Vector[11] = 14.56
Vector[12] = 17.32
Vector[13] = 20.32
Vector[14] = 23.56
Vector[15] = 27.04
Vector[16] = 30.76
Vector[17] = 34.72
Vector[18] = 38.92
Vector[19] = 43.36
Vector[20] = 47.92
```

```

// Code for void gradientFunc()

void gradientFunc(double myFunc(const double x), double x[], double dydx[], int m)
{

    // The first point: Three-point forward difference
    dydx[0] = (-3 * myFunc(x[0]) + 4 * myFunc(x[1]) - myFunc(x[2])) / (x[2] - x[0]);

    // In the middle: Two-point central difference
    for (int i = 1; i < m - 1; i++) {
        dydx[i] = (myFunc(x[i + 1]) - myFunc(x[i - 1])) / (x[i + 1] - x[i - 1]);
    }

    // The last point: Three-point backward for the last point
    dydx[m - 1] = (myFunc(x[m - 3]) - 4 * myFunc(x[m - 2]) + 3 * myFunc(x[m - 1])) / (x[m - 1] - x[m - 3]);
}

```

3. Create a function for 2nd order differentiation using discrete points

- Create a function for numerical differentiation from a set of data

```
void acceleration(double x[], double y[], double dy2dx2[], int m)
```

```
// First Point:    Four-point forward difference  (see APPENDIX)
```

```
// Mid Points:    Three-point central difference
```

```
// End Point:     Four-point backward difference
```

- Check your function with the test equation

“ $y=x^3$ ” at $x=0:0.2:4$

```
// show your output
```

```
*****
|                                     |
|                               PART 3. |
|                                     |
*****
Vector[0] = -0.00
Vector[1] = 1.20
Vector[2] = 2.40
Vector[3] = 3.60
Vector[4] = 4.80
Vector[5] = 6.00
Vector[6] = 7.20
Vector[7] = 8.40
Vector[8] = 9.60
Vector[9] = 10.80
Vector[10] = 12.00
Vector[11] = 13.20
Vector[12] = 14.40
Vector[13] = 15.60
Vector[14] = 16.80
Vector[15] = 18.00
Vector[16] = 19.20
Vector[17] = 20.40
Vector[18] = 21.60
Vector[19] = 22.80
Vector[20] = 24.00

Press any key to continue . . . |
```

```

// Code for void acceleration ()

void acceleration(double x[], double y[], double dy2dx2[], int m) {

    double h = x[1] - x[0];

    // First Point: Four-point forward difference
    dy2dx2[0] = (2 * y[0] - 5 * y[1] + 4 * y[2] - y[3]) / power(h, 2.0);

    // Mid Point: Three-point central difference
    for (int i = 1; i < m - 1; i++) {
        dy2dx2[i] = (y[i - 1] - 2 * y[i] + y[i + 1]) / power(h, 2.0);
    }

    // End Point: Four-point backward difference
    dy2dx2[m - 1] = (-y[m - 4] + 4 * y[m - 3] - 5 * y[m - 2] + 2 * y[m - 1]) /
power(h, 2.0);
}

```


Appendix

<i>Second Derivative</i>		
Method	Formula	Truncation Error
Three-point forward difference	$f''(x_i) = \frac{f(x_i) - 2f(x_{i+1}) + f(x_{i+2}))}{h^2}$	$O(h)$
Four-point forward difference	$f''(x_i) = \frac{2f(x_i) - 5f(x_{i+1}) + 4f(x_{i+2}) - f(x_{i+3}))}{h^2}$	$O(h^2)$
Three-point backward difference	$f''(x_i) = \frac{f(x_{i-2}) - 2f(x_{i-1}) + f(x_i))}{h^2}$	$O(h)$
Four-point backward difference	$f''(x_i) = \frac{-f(x_{i-3}) + 4f(x_{i-2}) - 5f(x_{i-1}) + 2f(x_i))}{h^2}$	$O(h^2)$
Three-point central difference	$f''(x_i) = \frac{f(x_{i-1}) - 2f(x_i) + f(x_{i+1}))}{h^2}$	$O(h^2)$
Five-point central difference	$f''(x_i) = \frac{-f(x_{i-2}) + 16f(x_{i-1}) - 30f(x_i) + 16f(x_{i+1}) - f(x_{i+2}))}{12h^2}$	$O(h^4)$